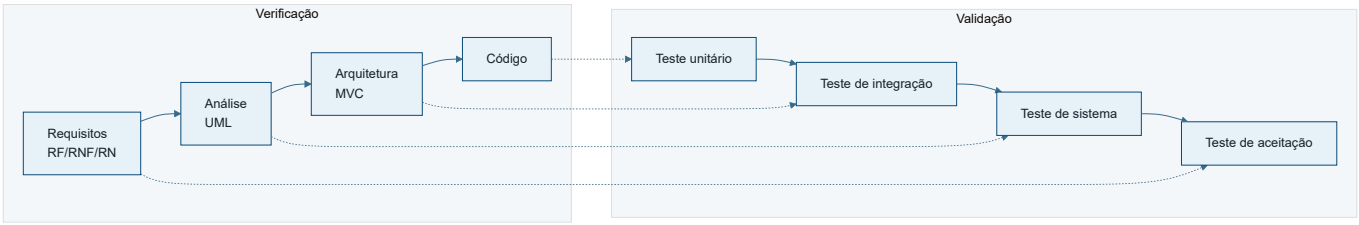
Plano de testes — Verificação e validação

1. Estratégia (Modelo V)



Nível do V	O que é verificado/validado	Como	Onde
Código ↔ Teste unitário	Funções e regras isoladas (conversões, CPF, placa, regras do Model, validação JS)	pytest e `node:test` automatizados	`tests/test_unitario_util.py`, `tests/test_unitario_models.py`, `tests/frontend/`
Arquitetura ↔ Teste de integração	Controller + Model + banco + templates juntos, via HTTP	`app.test_client()` do Flask com SQLite em memória	`tests/test_controllers.py`
Análise ↔ Teste de sistema	Fluxos completos dos casos de uso (UC04 → UC10 → UC11)	Cenários de ponta a ponta automatizados + roteiro manual	`tests/test_integracao.py` e seção 4
Requisitos ↔ Teste de aceitação	O sistema atende quem vai usar?	5 testadores externos com formulário padronizado	`03-formulario-teste-usuarios.md` e `laudos/`

Atividades de verificação (sem executar o sistema)

- Revisão do código contra os requisitos e as regras de negócio (tabela de rastreabilidade abaixo).
- Conferência de que o script `database/schema.sql` bate com os Models (gerado a partir do SQLAlchemy).
- Revisão dos diagramas UML contra as rotas e métodos implementados.

2. Ambiente

Item	Configuração
SO	Windows 11
Python	3.11 · Flask 3.0.3 · SQLAlchemy 2.0.36 · pytest 8.3.3
Node.js	24 (somente testes de front-end)
Banco de teste	SQLite em memória, recriado a cada teste (isolamento total)
Navegadores (aceitação)	Chrome e celular Android/iOS

3. Casos de teste e rastreabilidade

CT	Descrição	Entrada	Resultado esperado	Requisito/Regra	Teste automatizado
CT01	Catálogo exibe veículos disponíveis com preço formatado	Veículo cadastrado	"R$ 72.900,00", "35.000 km"	RF01	`test_catalogo_lista_veiculo_com_preco_formatado`
CT02	Catálogo oculta vendidos	Veículo VENDIDO	Não aparece; detalhe dá 404	RN03	`test_catalogo_nao_exibe_vendidos`, `test_detalhe_inexistente_ou_vendido_retorna_404`
CT03	Filtros combinados	preço, combustível, câmbio, texto	Apenas os veículos que atendem	RF02	`test_catalogo_filtros_combinados`
CT04	Ordenação por preço	menor/maior preço	Ordem correta	RF03	`test_catalogo_ordenacao_por_preco`

CT05	Parâmetros inválidos na URL	`preco_min=abc&pagina=999`	Página carrega (200)	RNF07	`test_catalogo_parametros_invalidos_nao_quebram`
CT06	Placa mascarada na vitrine	Detalhe do veículo	"*-23", sem placa completa	RNF12	`test_detalhe_do_veiculo_oculta_placa`
CT07	Enviar proposta válida	Dados corretos	Proposta PENDENTE + mensagem de sucesso	RF05	`test_enviar_proposta_valida`
CT08	Proposta inválida	e-mail/telefone errados	HTTP 422, erros nos campos, dados mantidos	RF05, RNF07	`test_enviar_proposta_invalida_mostra_erros_e_mantem_dados`
CT09	Proposta para veículo vendido	Veículo VENDIDO	Recusada	RN03	`test_proposta_para_veiculo_vendido`
CT10	Login inválido	Senha errada	401 + mensagem genérica	RF06	`test_login_invalido`
CT11	Painel sem login	GET /admin/veiculos/	Redireciona ao login	RNF06	`test_painel_exige_login`
CT12	Open redirect	`?proximo=//site-malicioso.com`	Ignorado	RNF04	`test_login_ignora_redirecionamento_externo`
CT13	Cadastrar veículo com foto	PNG válido	Gravado, foto salva com nome UUID	RF07	`test_cadastrar_veiculo_com_foto`
CT14	Cadastro inválido	Placa "XX", preço 0	422 com mensagens	RN01, RN02	`test_cadastrar_veiculo_invalido_retorna_422`
CT15	Placa duplicada	Mesma placa	Erro no campo placa	RN02	`test_veiculo_placa_duplicada`
CT16	Placas antiga e Mercosul	ABC1234, abc-1d23	Aceitas e normalizadas	RN02	`test_veiculo_aceita_placas_antiga_e_mercosul`
CT17	Ano do modelo incoerente	fab. 2021, modelo 2023	Erro	RN04	`test_veiculo_ano_modelo_coerente`
CT18	Upload malicioso	PHP renomeado para .png; .exe	Rejeitado	RNF05	`test_cadastrar_veiculo_rejeita_arquivo_que_nao_e_imagem`, `..._rejeita_extensao`
CT19	Editar veículo	Novo preço/cor	Atualizado	RF08	`test_editar_veiculo`
CT20	Editar/excluir vendido	Veículo VENDIDO	Bloqueado	RN09	`test_editar_veiculo_vendido_bloqueado`, `test_veiculo_vendido_nao_pode_ser_editado_nem_excluido`
CT21	Vendedor tenta excluir	Perfil VENDEDOR	403	RF09, RNF06	`test_vendedor_nao_pode_excluir_veiculo`
CT22	Marca duplicada / em uso	"CHEVROLET", marca com carro	Erro / bloqueio	RN10, RF11	`test_marca_duplicada_mostra_erro`, `test_excluir_marca_em_uso`
CT23	Aceitar proposta	Proposta pendente	ACEITA + veículo RESERVADO	RN06	`test_aceitar_proposta_reserva_veiculo`
CT24	Segunda proposta aceita	Duas propostas	Segunda bloqueada	RN06	`test_somente_uma_proposta_aceita_por_veiculo`
CT25	Recusar proposta aceita	Proposta ACEITA	RECUSADA + veículo DISPONÍVEL	RN07	`test_recusar_proposta_aceita_libera_veiculo`
CT26	Registrar venda	Proposta aceita + CPF	VENDIDO, outras propostas recusadas	RN08	`test_venda_marca_veiculo_como_vendido_e_recusa_outras_propostas`

CT27	Vender duas vezes	Mesmo veículo	Bloqueado	RN09	`test_veiculo_nao_pode_ser_vendido_duas_vezes`
CT28	CPF inválido / de outra pessoa	CPF errado ou já usado	Erro	RN12	`test_venda_exige_cpf_valido`, `test_cliente_cpf_de_outra_pessoa`
CT29	CSRF	POST sem token	400 "Sua sessão expirou"	RNF04	`test_csrf_bloqueia_post_sem_token`
CT30	XSS	`<script>` no nome	Exibido escapado	RNF04	`test_campos_sao_escapados_contra_xss`
CT31	Senha com hash	Usuário criado	Hash ≠ senha	RNF03	`test_usuario_senha_e_armazenada_com_hash`
CT32	Fluxo completo	Cadastro → proposta → aceite → venda	Estado final consistente	UC04, UC06, UC10, UC11	`test_fluxo_completo_cadastro_proposta_venda`
CT33	Validação no navegador	Placa, e-mail, telefone, CPF, anos, valor	Mensagens corretas	RNF07	`tests/frontend/validacao.test.js`
CT34	Máscaras e moeda (JS)	Dígitos digitados	(11) 98765-4321 · 529.982.247-25 · R$ 85.000,00	RNF07	`tests/frontend/validacao.test.js`

4. Roteiro de teste manual de sistema (navegador)

#	Passo	Resultado esperado	OK?
M01	Abrir o catálogo no celular	Layout em uma coluna, menu "Menu" abre/fecha	☐
M02	Digitar e-mail inválido no formulário de proposta e sair do campo	Mensagem vermelha imediata	☐
M03	Digitar telefone só com números	Máscara (11) 98765-4321 aplicada	☐
M04	Escolher foto de 8 MB no cadastro	Erro "A imagem deve ter no máximo 5 MB" antes do envio	☐
M05	Escolher foto válida	Prévia aparece	☐
M06	Clicar duas vezes rápido em "Cadastrar veículo"	Só um veículo criado	☐
M07	Clicar em "Excluir" e cancelar a confirmação	Nada é excluído	☐
M08	Navegar só com Tab/Enter	Foco visível e todos os botões acessíveis	☐
M09	Acessar `/admin` em aba anônima	Redireciona ao login com aviso	☐
M10	Acessar `/veiculos/99999`	Página 404 amigável	☐

5. Critérios de aceite

- 100% dos testes automatizados passando.
- Cobertura do back-end ≥ 80%.
- Nenhum defeito de severidade **alta** aberto após a rodada com os 5 testadores.
- Todo defeito relatado registrado em `04-registro-de-correcoes.md`, corrigido ou justificado.

Relatório de testes automatizados

Data da execução inicial: 14/10/2026 **Reexecução de confirmação:** 11/11/2026 **Versão testada:** código atual da branch `main` do repositório
Evidências (saída completa): `evidencias/resultado-pytest.txt` e `evidencias/resultado-frontend.txt`

1. Resumo

Suíte	Nível do Modelo V	Testes	Passaram	Falharam
`test_unitario_util.py`	Unitário	32	32	0
`test_unitario_models.py`	Unitário (regras de negócio)	54	54	0
`test_controllers.py`	Integração (HTTP + banco + templates)	39	39	0
`test_integracao.py`	Sistema (fluxos completos + comando seed)	3	3	0
`tests/frontend/validacao.test.js`	Unitário (front-end)	13	13	0
Total		**141**	**141**	**0**

Os 128 testes Python rodaram com `-W error::Warning`, ou seja, qualquer aviso seria tratado como falha. Na reexecução de confirmação de 11/11/2026, os 128 testes Python e os 13 testes JavaScript passaram novamente.

2. Cobertura de código do back-end

Módulo	Cobertura
`app/models/*` (regras de negócio)	94% a 100%
`app/controllers/*`	82% a 100%
`app/seguranca.py`	96%
`app/servicos/armazenamento.py`	76% (sem cobertura: só os ramos de falha de disco ao remover arquivos)
Total	**93%** (meta: 80%)

3. Defeitos encontrados pelos testes durante o desenvolvimento

A suíte encontrou defeitos reais, que foram corrigidos antes da entrega:

#	Defeito	Como foi detectado	Correção
D01	No JavaScript, o ano do modelo **vazio** era convertido para `0` e mostrava a mensagem errada ("deve ser igual ao ano de fabricação…") em vez de "Informe o ano do modelo."	`validacao.test.js` › "ano do modelo igual ou um ano após a fabricação"	`validarAnoModelo` passou a tratar texto vazio antes de converter para número.
D02	Ao registrar a venda, o SQLAlchemy emitia aviso de objeto `Venda` fora da sessão durante o *autoflush*. Havia risco de a venda não ser associada ao cliente/vendedor em alguns cenários.	Execução com `-W error::Warning` em `test_venda_*`	`db.session.add(venda)` movido para antes de alterar o status das propostas.

4. Validação de segurança (resumo)

Verificação	Resultado
POST sem token CSRF	Bloqueado (400) ✓
POST com token CSRF válido	Aceito ✓
`<script>` em campo de texto	Exibido escapado ✓
Arquivo PHP com extensão `.png`	Rejeitado pela assinatura binária ✓
Redirecionamento externo após login	Ignorado ✓
Vendedor executando ação de admin	403 ✓
Cabeçalhos `X-Frame-Options` / `nosniff`	Presentes ✓

5. Teste manual em navegador (verificação feita pelo desenvolvedor)

Executado com a aplicação real (`flask run`, CSRF ligado, banco com `seed`):

Verificação	Resultado
Todas as rotas do painel respondem 200 após login	✓
Login sem token CSRF retorna 400	✓
Filtro "Híbrido" retorna apenas o Corolla Cross	✓
Validação em tempo real do e-mail no formulário de proposta	✓ mensagem exibida ao sair do campo
Máscara de telefone ao digitar `11987654321`	✓ `(11) 98765-4321`
Erros de JavaScript no console	Nenhum

6. Como reproduzir

```
 pip install -r requirements.txt
pytest -vv --cov=app
node --test "tests/frontend/*.test.js"
```

7. Teste de aceitação (usuários)

Está descrito em `03-formulario-teste-usuarios.md`. Os laudos dos 5 testadores ficam em `laudos/` e as correções em `04-registro-de-correcoes.md`.

Formulário de teste de aceitação (5 testadores)

Como conduzir

1. Escolha **5 pessoas**: colegas de curso ou profissionais da área.
2. Deixe o sistema acessível:

- **Opção A (mais simples):** publique em um serviço gratuito (Render, PythonAnywhere) e envie o link.
- **Opção B:** o testador clona o repositório e roda localmente seguindo o `README.md`.

1. Envie a cada testador:

- o link do sistema;
- os logins de demonstração (admin e vendedor);
- o **roteiro de tarefas** (seção 2);
- o **formulário** (seção 3), no Google Forms ou neste modelo em Word/PDF.

1. Peça prints das telas com erro.
2. Salve cada resposta em `laudos/` como `laudo-01-nome.pdf` … `laudo-05-nome.pdf`.

> **Importante:** o PDF da disciplina pede que as falhas **não sejam escondidas**. Registre tudo o que os testadores relatarem, inclusive o que você decidir não corrigir (com justificativa).

Roteiro de tarefas para o testador

Como visitante

1. Encontre um carro automático com preço até R$ 130.000.
2. Abra os detalhes de um carro e envie uma proposta. Tente antes enviar com um e-mail errado.

Como vendedor (`vendedor@autovenda.com` / `Vende@123`)

1. Cadastre um veículo com foto.
2. Tente cadastrar outro veículo com a **mesma placa**.
3. Aceite a proposta que você enviou na tarefa 2 e verifique o catálogo.
4. Registre a venda desse veículo.

Como administrador (`admin@autovenda.com` / `Admin@123`)

1. Cadastre uma marca nova e tente excluir uma marca que tenha carros.
2. Exclua o veículo cadastrado na tarefa 3.

Livre

1. Use o sistema pelo celular por alguns minutos.

Modelo do laudo de teste

Sistema: AutoVenda · **Versão/link testado:** _____

Campo	Resposta
Nome de quem testou	
Formação / área	☐ Colega de curso ☐ Profissional de TI ☐ Outro: ____
Data do teste	___/___/_____
Dispositivo e navegador	ex.: notebook + Chrome; celular Android + Chrome

1. O que testou e FUNCIONOU

Funcionalidade	Observação

2. O que testou e NÃO FUNCIONOU

Funcionalidade	O que aconteceu	O que deveria acontecer / sugestão de correção	Print (anexo nº)	Gravidade (Alta/Média/Baixa)

3. Funcionalidades NÃO TESTADAS

Funcionalidade	Motivo (não encontrou, faltou tempo, não existe no sistema, deu erro antes…)

4. Avaliação geral (1 = ruim, 5 = ótimo)

Critério	1	2	3	4	5
Facilidade de uso	☐	☐	☐	☐	☐
Clareza das mensagens de erro	☐	☐	☐	☐	☐
Aparência	☐	☐	☐	☐	☐
Uso no celular	☐	☐	☐	☐	☐
Velocidade	☐	☐	☐	☐	☐

5. Sugestões de melhoria

_____

6. Anexos (prints)

Anexo 1: _____ · Anexo 2: _____

Assinatura do testador: _____

Versão Google Forms

Crie o formulário com estas perguntas, na ordem:

1. Nome completo *(resposta curta, obrigatória)*
2. Formação / área *(múltipla escolha: Colega de curso, Profissional de TI, Outro)*
3. Data do teste *(data, obrigatória)*
4. Dispositivo e navegador usados *(resposta curta)*
5. Quais funcionalidades você testou e FUNCIONARAM? Descreva cada uma. *(parágrafo, obrigatória)*
6. Quais funcionalidades você testou e NÃO FUNCIONARAM? Descreva o que aconteceu e o que deveria acontecer. *(parágrafo)*
7. Gravidade do principal problema encontrado *(múltipla escolha: Alta (impede o uso), Média (atrapalha), Baixa (estético), Não encontrei problemas)*
8. Envie prints das telas com erro *(upload de arquivo, até 5 imagens)*
9. Quais funcionalidades você NÃO testou e por quê? *(parágrafo)*
10. Facilidade de uso *(escala 1–5)*
11. Clareza das mensagens *(escala 1–5)*
12. Aparência *(escala 1–5)*
13. Uso no celular *(escala 1–5)*
14. Sugestões de melhoria *(parágrafo)*

Depois de coletar as 5 respostas, use **Respostas › ⋮ › Imprimir todas as respostas › Salvar como PDF** para gerar o PDF exigido na entrega.

Registro de correções e melhorias pós-teste

> Preencha uma linha para **cada** item relatado nos laudos, inclusive os que não forem corrigidos. A coluna "Commit" liga a correção ao código (rastreabilidade requisito ↔ teste ↔ código).

Correções vindas dos testadores

#	Laudo	Problema/sugestão relatado	Gravidade	Decisão	O que foi feito	Commit	Teste que cobre
C01	laudo-01			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C02	laudo-02			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C03	laudo-03			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C04	laudo-04			☐ Corrigido ☐ Melhoria futura ☐ Não procede			
C05	laudo-05			☐ Corrigido ☐ Melhoria futura ☐ Não procede			

Correções encontradas pelos testes automatizados (antes da rodada com usuários)

#	Problema	Correção	Teste
D01	Ano do modelo vazio gerava mensagem errada no navegador	Tratamento de campo vazio em `validarAnoModelo`	`tests/frontend/validacao.test.js`
D02	Aviso do SQLAlchemy (Venda fora da sessão no autoflush)	`session.add(venda)` antes de atualizar as propostas	`test_venda_marca_veiculo_como_vendido_e_recusa_outras_propostas`

Resumo da rodada de aceitação

Indicador	Valor
Testadores	5
Itens relatados	
Corrigidos	
Adiados (melhoria futura)	
Não procedentes	
Nota média de facilidade de uso	

Boas práticas ao corrigir

1. Para cada bug, escreva primeiro um teste que reproduz o problema (ele deve falhar).
2. Corrija o código até o teste passar.
3. Rode a suíte inteira (`pytest` e `node --test`).
4. Faça um commit por correção, citando o item: `git commit -m "C03: mensagem de placa mais clara"`.
5. Atualize esta tabela e o vídeo final.